

PROGRAMMING LANGUAGES.

CSE . . .

Programming Paradigms

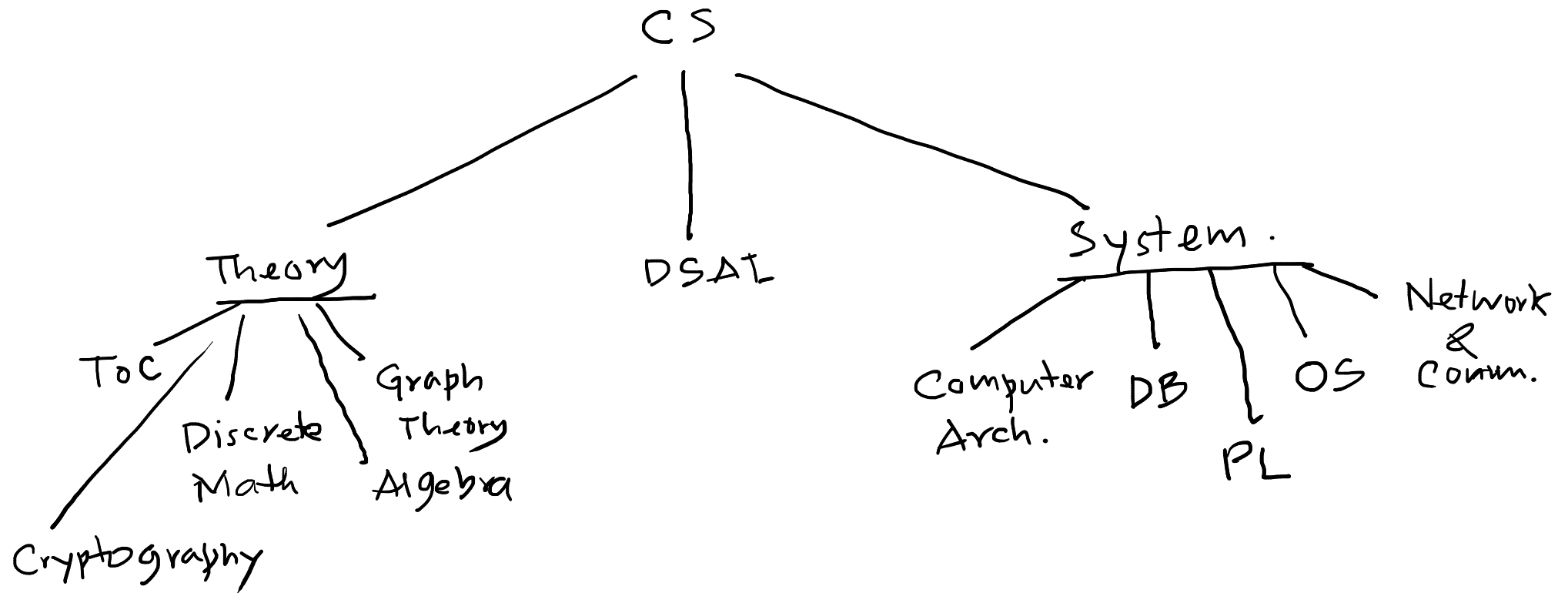
- Object-oriented paradigm - various
- Imperative programming - —
- ✓ - Functional paradigm - OCaml.
- Logic programming - Prolog.

PLDI - Programming Language Design and Implementation

- Syntax
- Semantics .

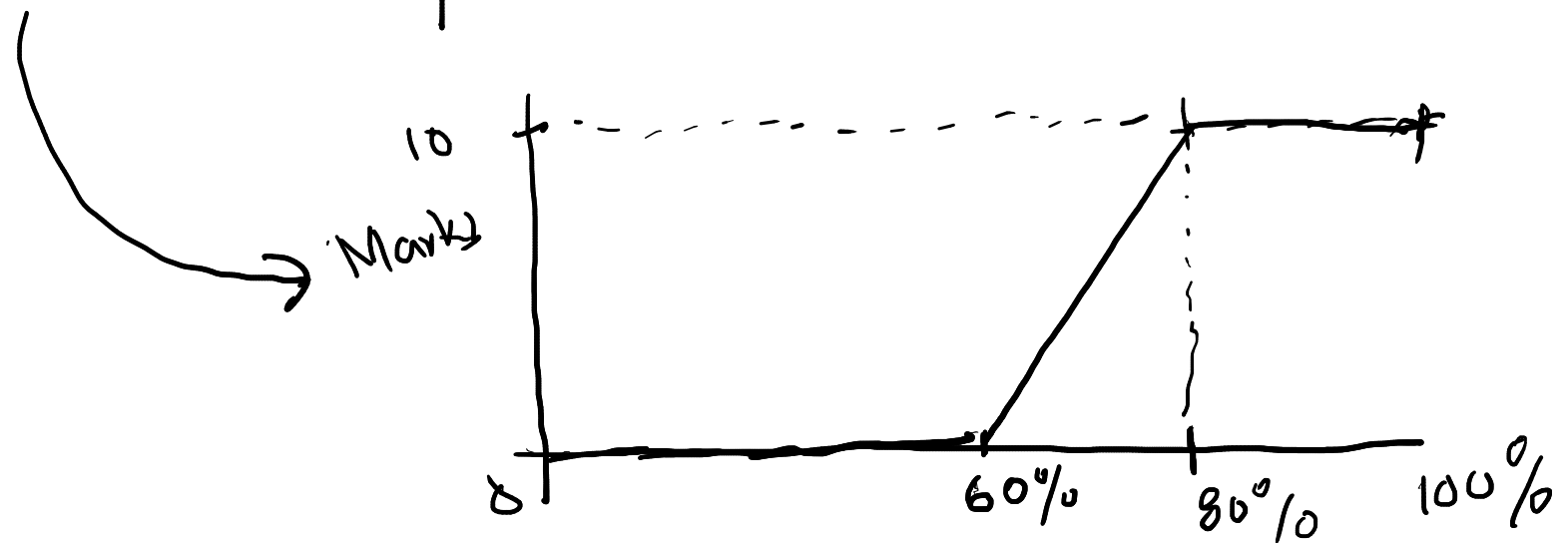
MOTIVATION

- Learning new languages
- Foundations of CS.
- Choice of PL or paradigm
- Getting a job
- Formal verification



LOGISTICS / MECHANICS

①	End-term	30
②	Mid-term	25
③	Project	25
④	Assignments	10
⑤	Attendance	10



PROJECT

- ① Teams - 2-3 persons
- ②
 - a Implementation
 - b Study
 - c Research

20/11/26.

Object Oriented Programming.

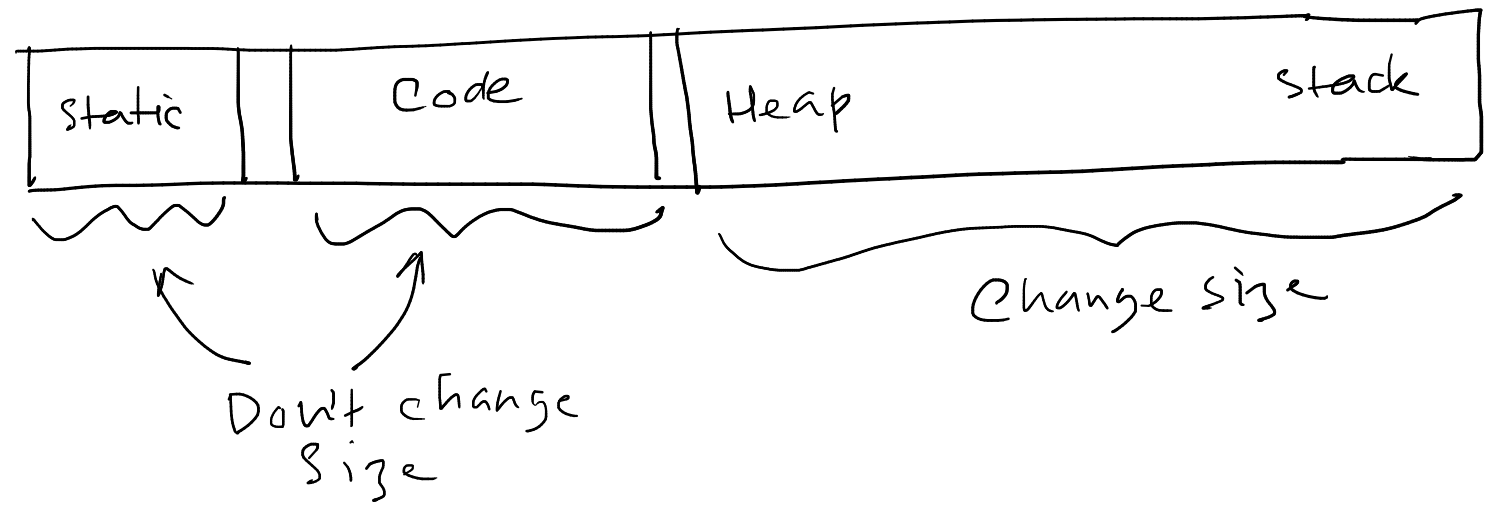
```
shape s = new Rectangle(10,5);
```

```
...
```

```
delete(s);
```


Memory Segments

- Stack
- Heap
- Code
- Static



Stack

```
def factorial(n):
```

```
    if (n == 1):
```

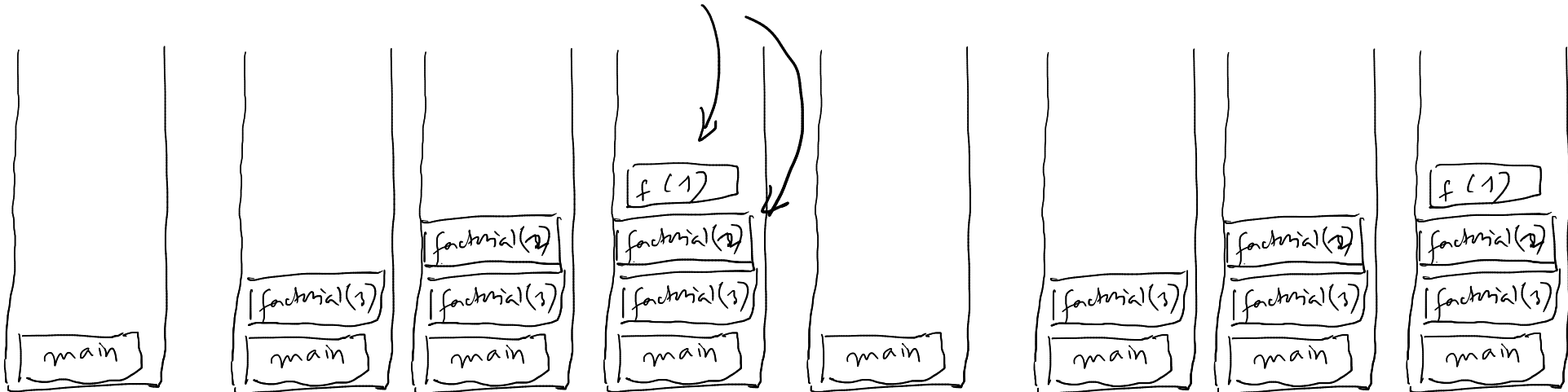
```
        return 1
```

```
    else:
```

```
        return n * factorial(n-1)
```

-
factorial(3)

Stack frame
Activation records.



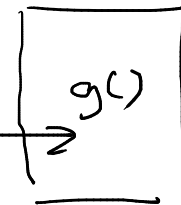
Call stack
Program stack

Activation Record

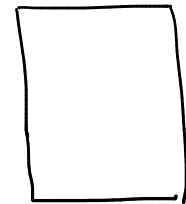
- ① Local variables including formal parameter
- ② Return address
- ③ Return value

Caller procedure
Callee —||—

f



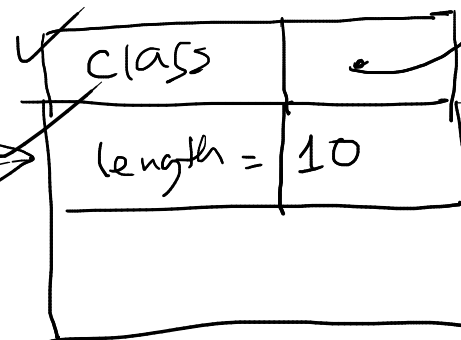
g



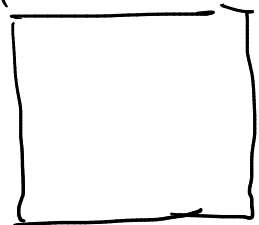
Heap

Shape *s = new Square(10);

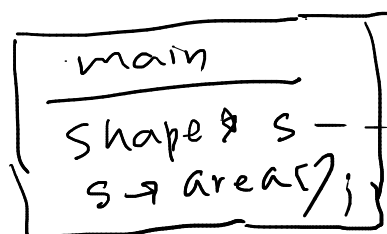
Heap



static
Square



Call stack



Static

method table
virtual table
v table

Code

Shape

area	
print	

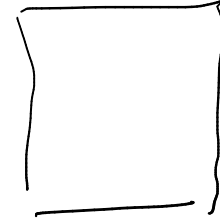
Square

area	
print	

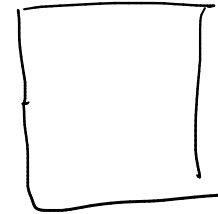
Rectangle:

area	
print	

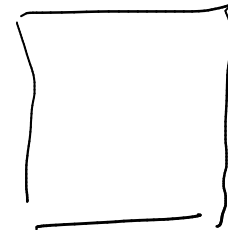
Square::area



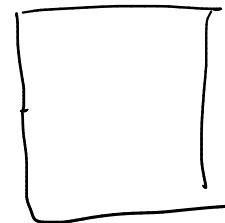
Square::print



Rectangle::area



Rectangle::print



Dynamic Dispatch

- ① Identify the object
- ② Identify method table
- ③ Identify the method (in code segment) whose pointer is in the offset corresponding to callee method (virtual) in the method table.

static dispatch.

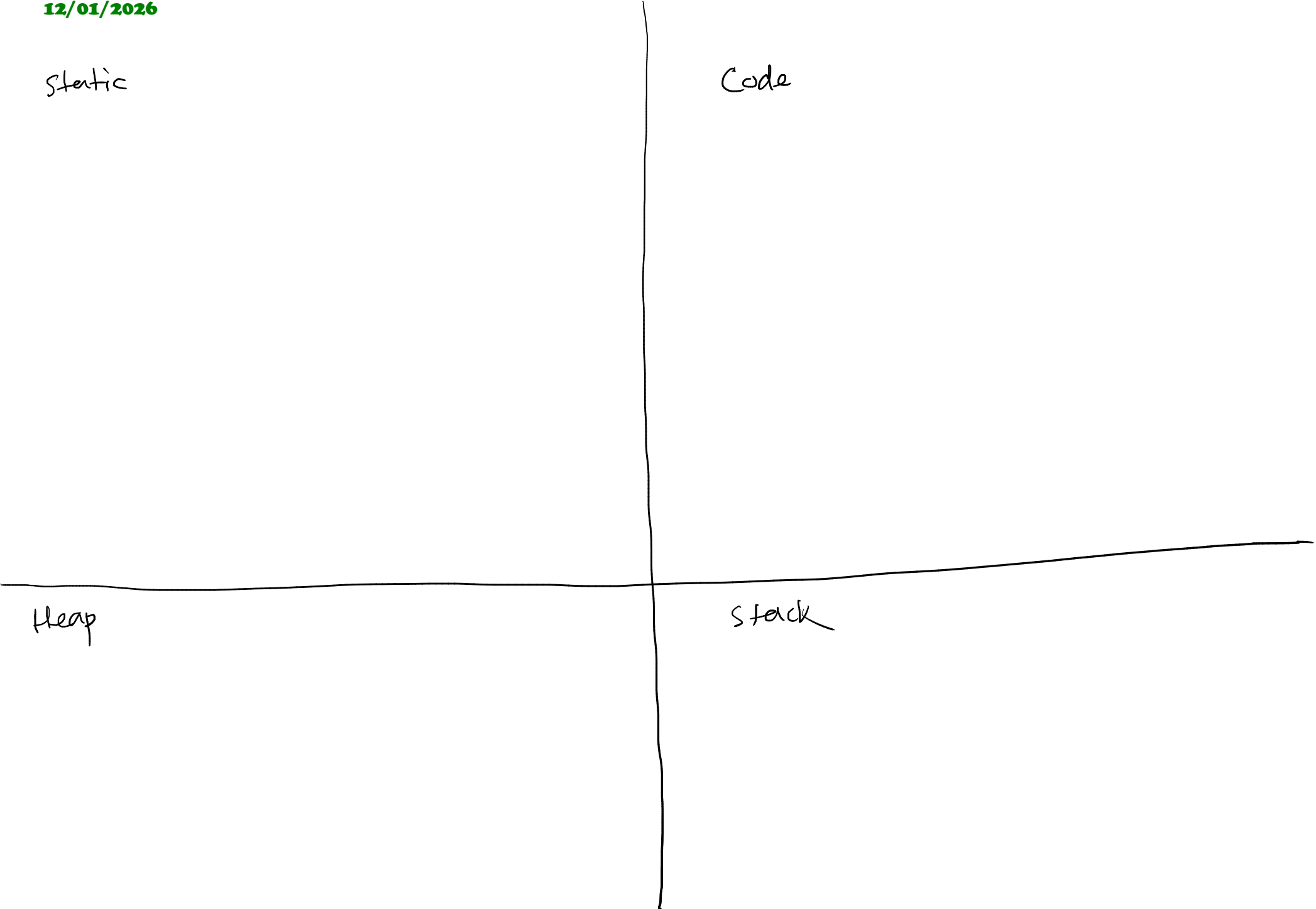
12/01/2026

static

Code

Heap

Stack

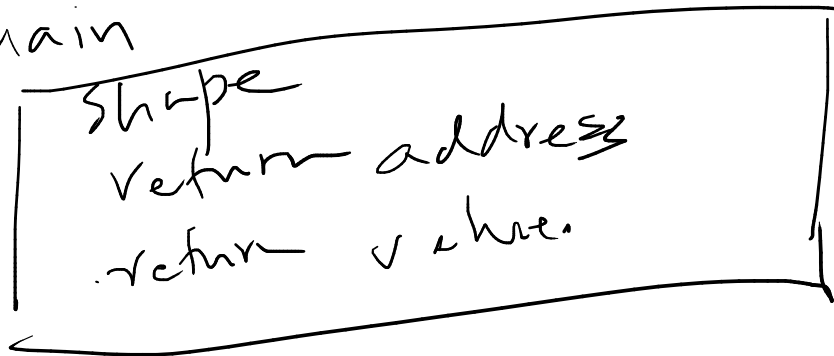


Details of the activation record can be shown separately.



Activation Record

main



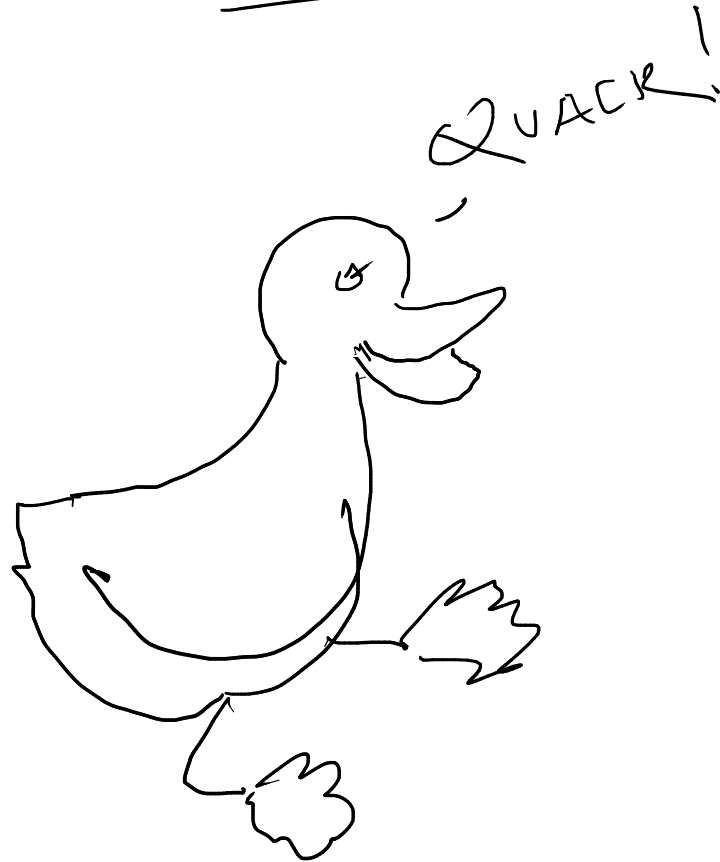
Reading Exercise

Method Resolution Ordering (MRO) .

If you walk like a duck
and you quack like a duck
then you ARE a duck.

DUCK TYPING

--str--



Reading exercises in OOP

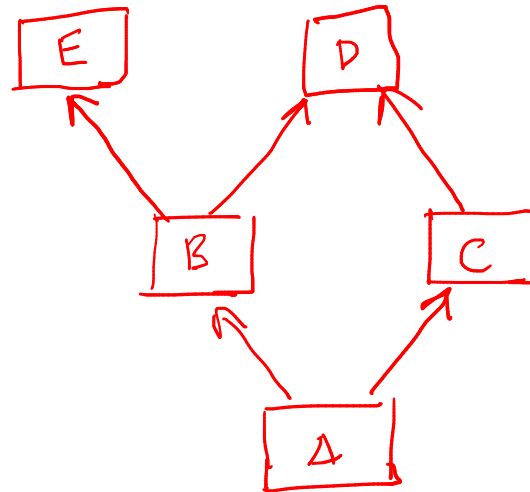
- ① Multimethod
- ② Double dispatch.

What? How? Why?

Multiple Inheritance

Dynamic Languages.

MRO



m

Static Languages

?

29/01/2026

Implementation of Dynamic Polymorphism in Presence of Multiple Inheritance

— Md. Ibrahim

Nested if

if $\langle \text{condition} \rangle$ then $\langle \text{expr1} \rangle$ else $\langle \text{expr2} \rangle$

if $\langle c1 \rangle$ then
 $\langle e1 \rangle$

else
 if $\langle c2 \rangle$ then
 $\langle e3 \rangle$
 else
 $\langle e4 \rangle$

Syntactic Sugar

Currying.

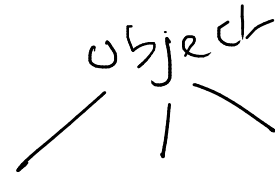
$f\ a_1\ a_2\ \dots$

$\text{add} : \text{int} \times \underset{\uparrow}{\text{int}} \longrightarrow \text{int}$

03/02/2026

classwork.

Write a find function such that find $e \in l$ returns true if $e \in l$ else returns false.



Erlang - WhatsApp.

B^*

$A^* a = \text{new } B.$

$\text{void } * m() \{ \dots \}$

$A^* a = m();$

$A^* a = \text{null}$

$B^* b = \text{null}$

A
 $\uparrow$
 B



Classwork

Transfer all code written today in REPL to a file.

- Compile and run it from shell using `ocaml command`
- Run it from REPL using `# use`

05/02/2026

Implement a polymorphic binary tree.

- n : gives the number of nodes.
- search : searches for a value.

Complete
Bintree

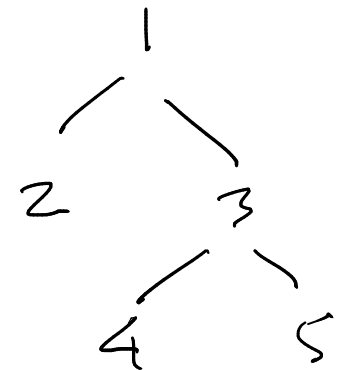
Internal Node (l, r)
Leaf

In Complete
Bintree

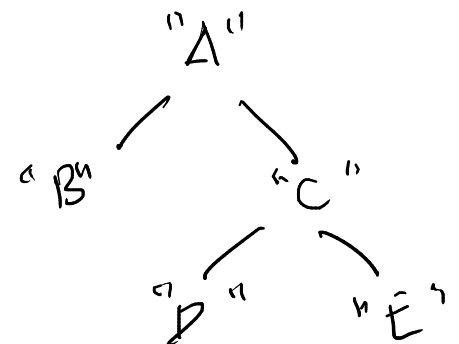
Internal Node (l, r)
l Internal Node (l)
r Internal Node (r)
Leaf

InternalNode $(l, r) \rightarrow$

$t_1 =$



$t_2 =$



Syntactic Restrictions on name

Must start with lowercase

- variables & function names

- types

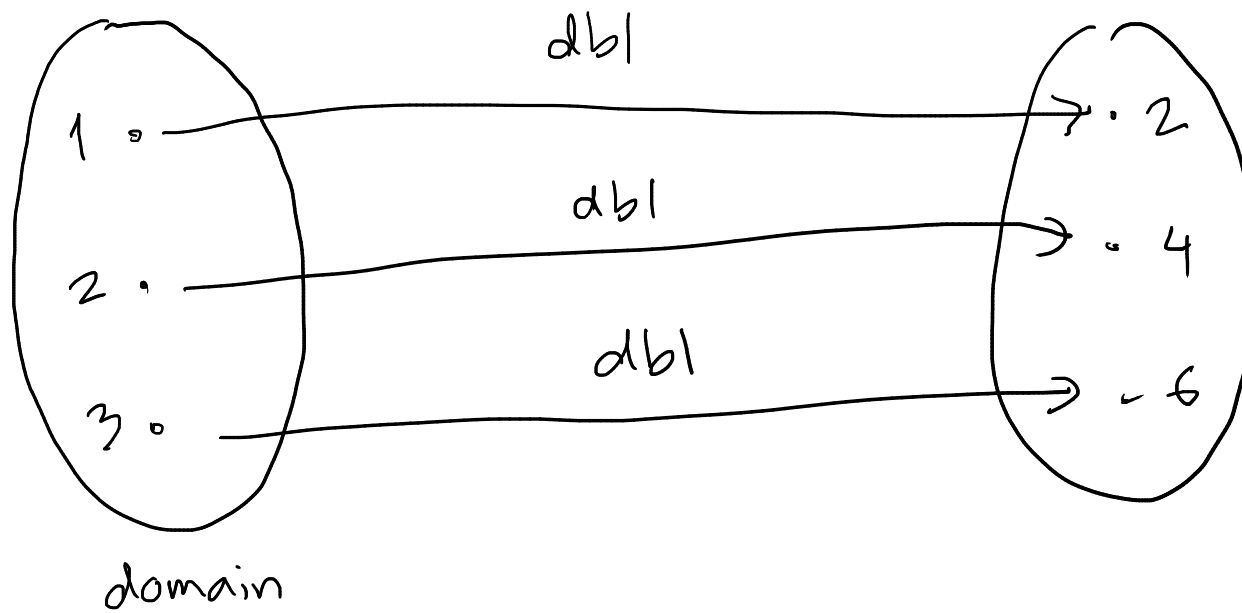
-

Must start with uppercase

- Constructors (invariants)

- Module names (to be discussed)

Map



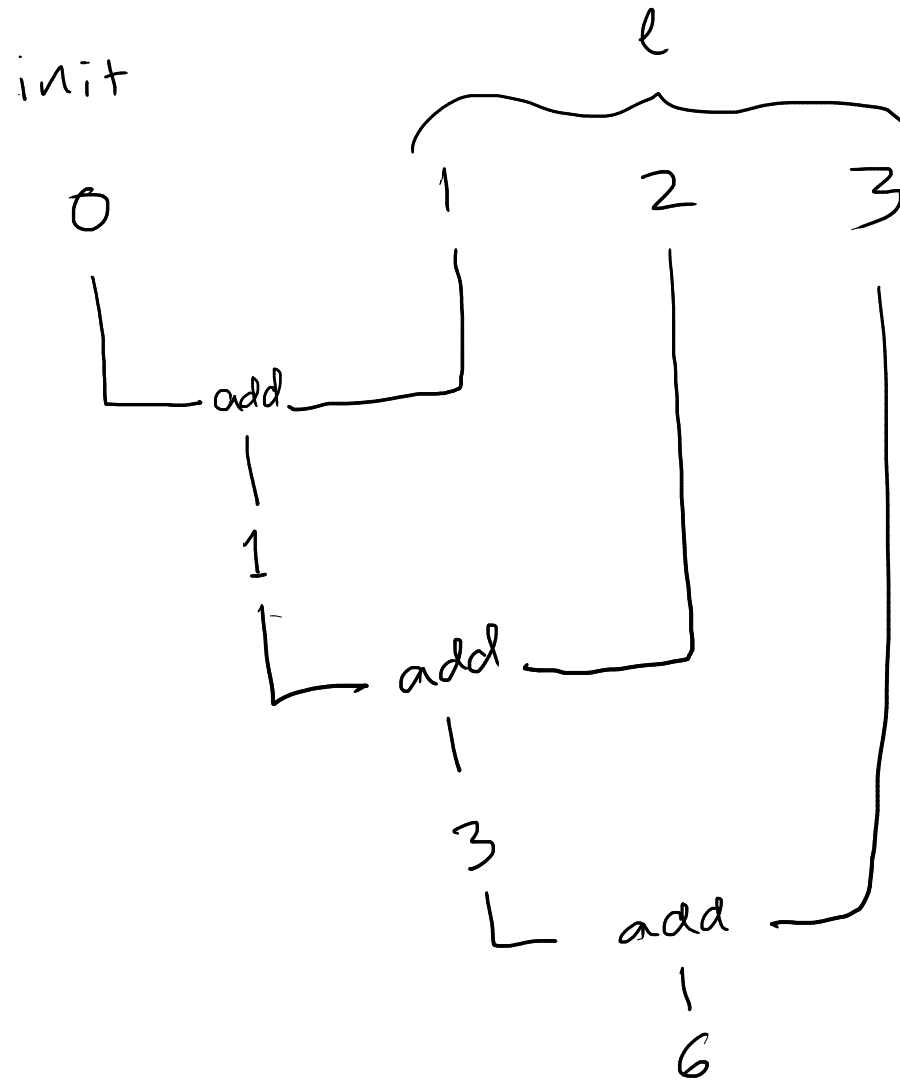
Fold (reduce)
fold-left

f

init

l

f = add



Classwork

A handwritten diagram of the standard deviation formula with several green annotations and numbered circles:

- sqrt**: A green arrow points to the square root symbol ($\sqrt{\quad}$).
- sumlist**: A green arrow points to the summation symbol ($\sum$).
- avg**: A green circle with the number 1 is connected by an arrow to the mean symbol (μ).
- 3**: A green circle with the number 3 is connected by an arrow to the squared term $(x - \mu)^2$.
- List.length**: A green circle containing $|X|$ is connected by an arrow to the denominator of the formula.

The formula itself is written as:

$$\sigma = \sqrt{\frac{\sum_{x \in X} (x - \mu)^2}{|X|}}$$

marks : float list

stddev

Define a function `pass_stddev` which computes the standard deviation of all the passing marks in a list `marks`.

use a combination of `map`, `filter` and `fold`.

let $x = l \pm z$ in

let $y = \underbrace{(\overset{\uparrow}{a})}_{\dots}$ in

let $z = y + x$

10/02/2026

Eager - OCaml, Java, Python

Lazy - Haskell

lazy and eager semantics

$g()$ ✓

$h()$

$f(g()) \Rightarrow$

let $x = g()$ in $f(x)$

let $f\ x\ y\ z =$

if $x = \text{true}$ then y

else z

delay / avoid unnecessary
computation by passing
functions.

$f\ \text{true}\ \underline{g()}\ \underline{h()}$

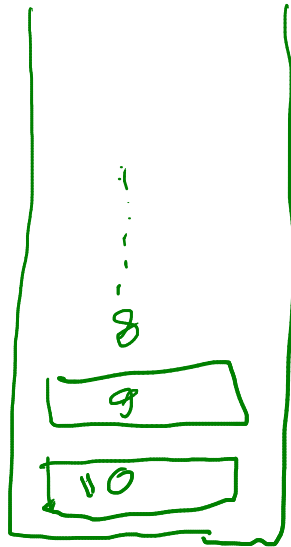
Thanks: functions encapsulating expression
with no parameters.

Tail calls and tail recursion

f

g()

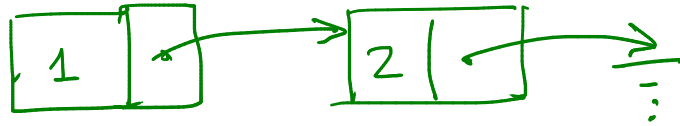
Tail call — A function call that's the last computation in a function.



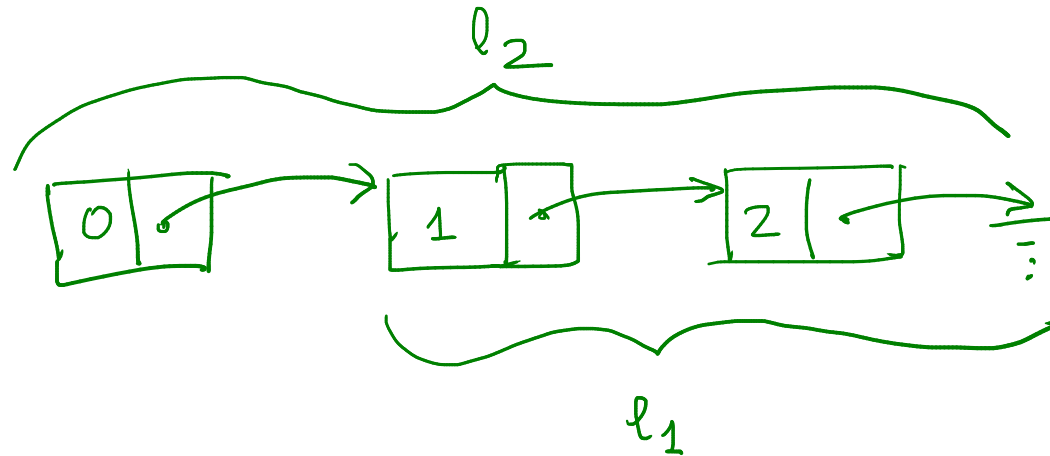
In a tail call, there's no need to preserve the caller's stack frame.

List construction ($::$) is not equivalent to append in Python.

$l_1 = [1, 2]$



$l_2 = 0 :: l_1$



Homework

Tail recursive implementations of.

- ① Filter
- ② Fold left
- ③ Fold right
- ④ List.reverse
- ⑤ List.append

12/02/2026

Implement a function `doublelist` that takes a list as parameter and returns a list with all elements doubled.

Use `List.map`.

Functions are first class objects.

First class objects. {
- Passed as parameter
- Returned as result of computation
- They can be stored in data-structures.

Functions are data!

Implement a function `pick-the-odds` that takes a list as parameter and returns a list with all elements odd.

Use `List.filter`, use partial application.

Derivative

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

$$h = 0.0001$$

17/02/2026.

Product types

variants are sum-types.

type wd = su | Mo | ... | sa.



Constructors / fields

What is UDT in which the no. of distinct types is the product of the # of values each field constitutes.

Structure → Record.
Classes

Lists

Tuples

Sets.

:

Activity

- ① Define a record type name with two fields `firstname` and `last name`.
- ② Define a function `fullname` which takes a record of name type and computes the full name.
- ③ Define a function `offspring : string → name → name` that gives the full name of offspring.

e.g. `let raj = {fn = "Raj" ; ln = "Kapoor"}`;;

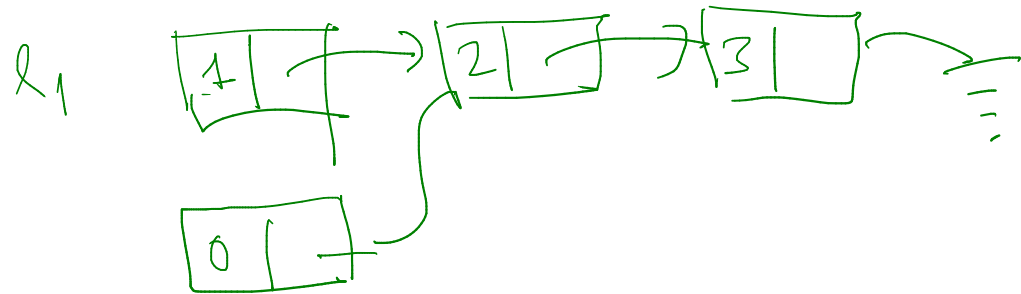
`let rishi = offspring "Rishi" raj;;`

(`fullname rishi = "Rishi Kapoor"` *)

12|03|25

let $l_1 = [1; 2; 3];$

let $l_2 = 0 :: (\text{List.tl } l_1);$



17/03/2026

s1 is a shirt.
s1's colour is red.

s2 is a shirt.
s2's colour is blue.

p1 is a pants.
p1's colour is green.

p2 is a pant.
p2's colour is violet.

skc wears s1 and p1.

ss wears s2 and p2.

red complements green.

yellow complements violet.

blue complements orange.

A person is considered well-dressed if he/she wears shirt and pants of complementary colour.

Is skc well-dressed?

Prolog Knowledge Base

Has two parts:

- Facts
- Rules

X is descendent of Y if

- Y is father of X.
- There exists Z whose descendent is X and Y is Z's father.

$\text{fact}(1, 1).$

In prolog:

$\text{factrial}(N, X)$

Logical condition for factorial-ness

$\text{/* } X \text{ is factrial of } N \text{ */}$

$\forall x.$ $x = \text{factorial}(n)$ if

$\exists x' \mid$ $x' = \text{factorial}(n')$

$\wedge$

$n' = n - 1$

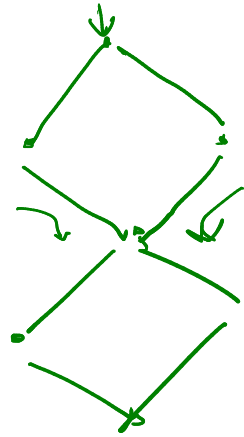
$\wedge$

$x = n \times x'$

$X = Y.$

$\forall X, Y \quad X = Y.$

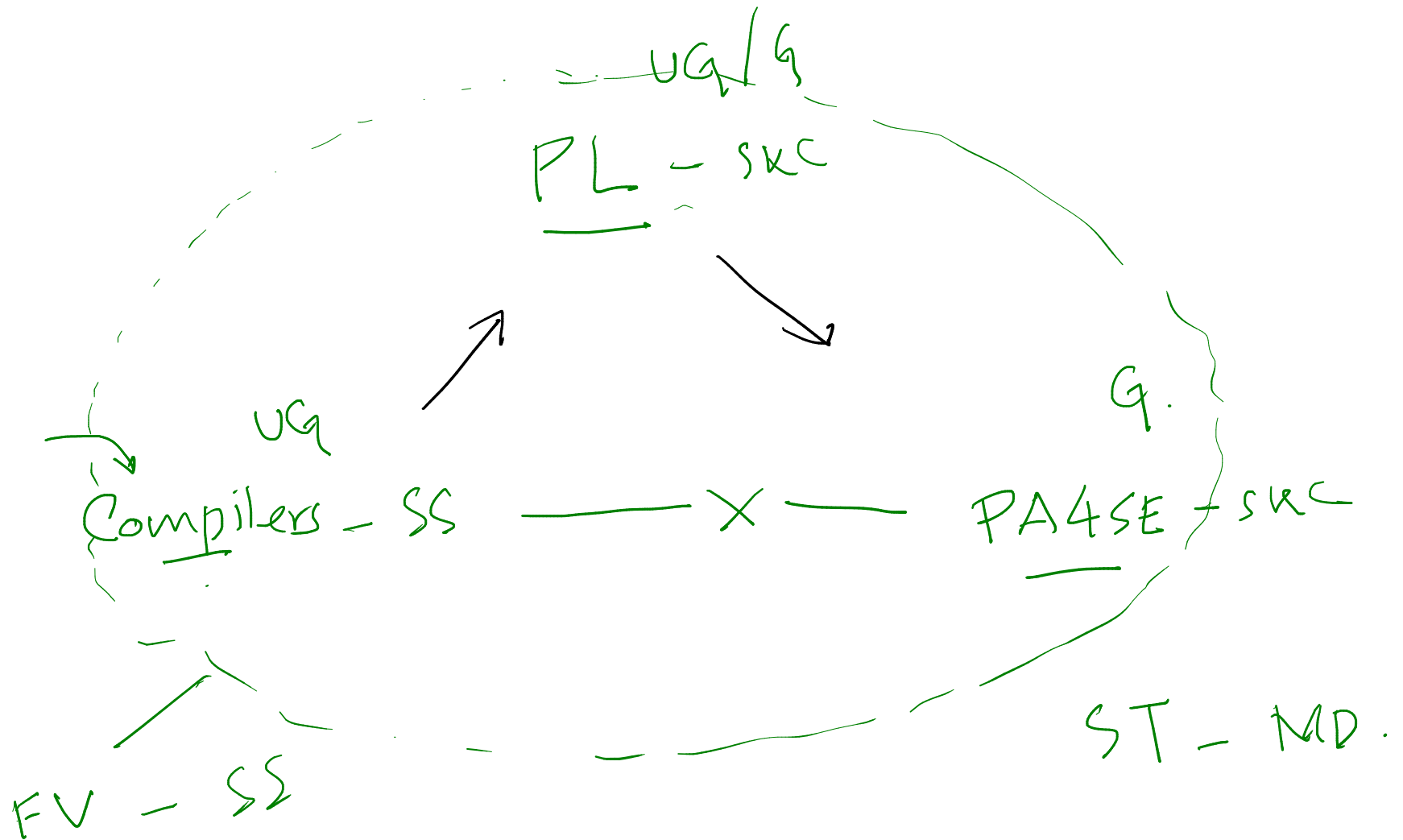
02/04/2026



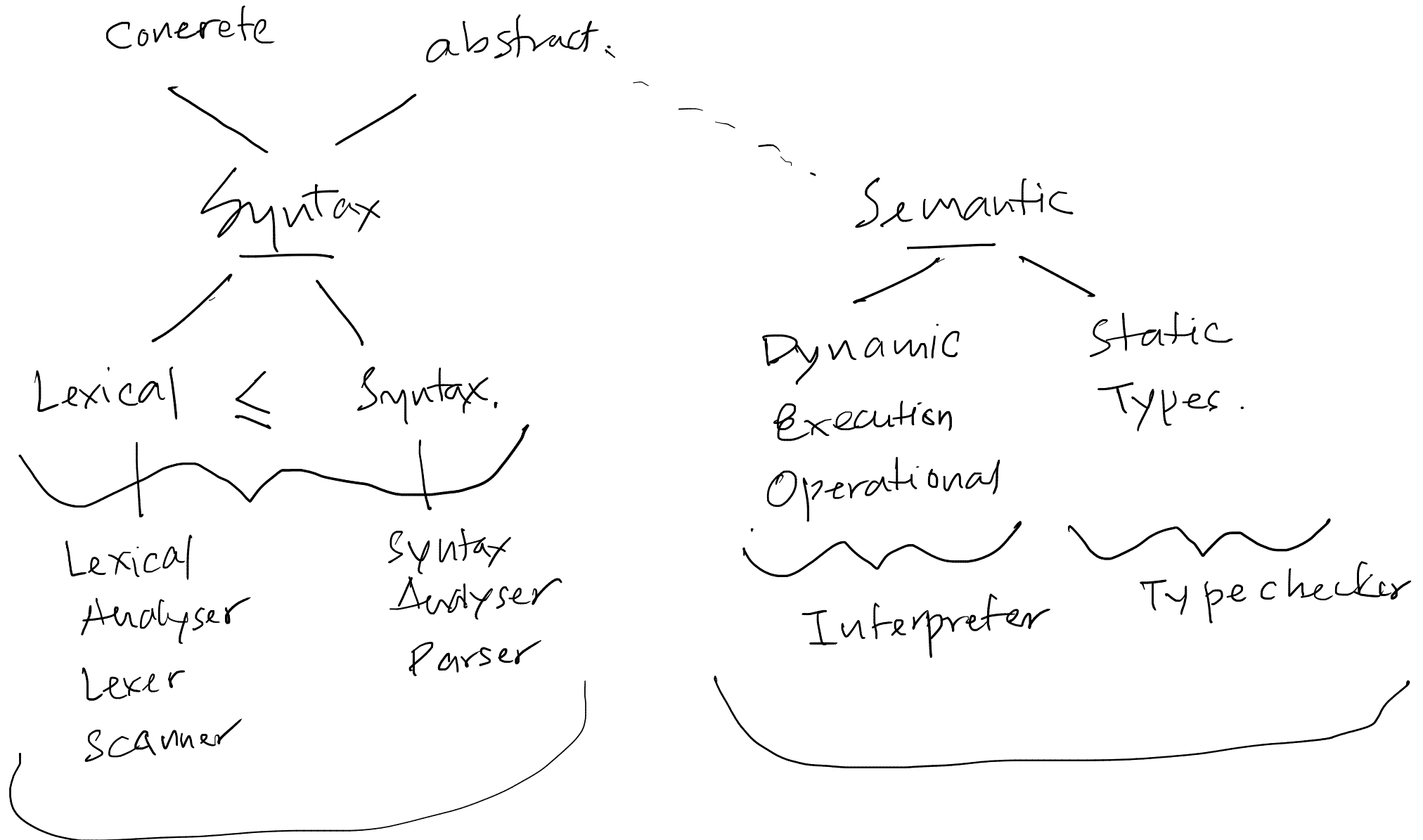
Programming Language Design and Implementation

PLDI.

Courses in PL



Description of Computer Languages.



23/04/2026.

Semantics

① Structural / static — Types

② Dynamic / execution semantics — Operational semantics.

Languages at play

- Input language
- Implementation language
- Specification language

Specification of Operational Semantics

- Declarative
- Precise/unambiguous.

Mathematical logic

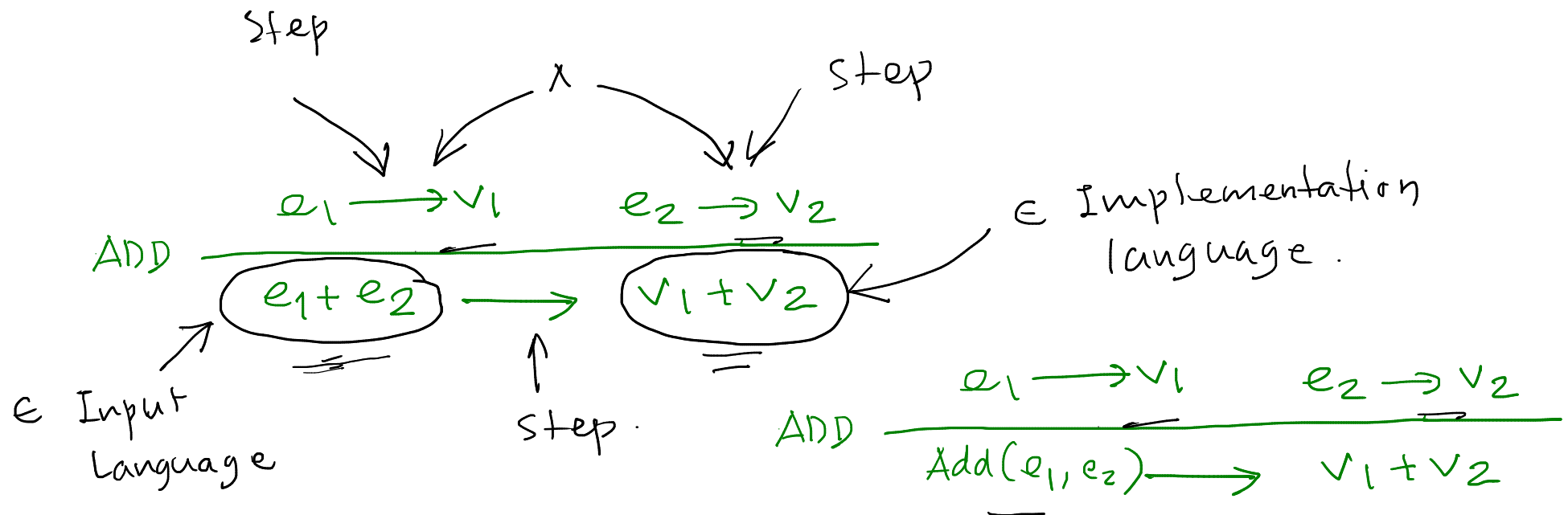
$$\frac{P \Rightarrow Q}{\text{ }} \equiv$$

$$\frac{P}{Q}$$

premise/antecedent

Rules of inference.

result/consequent



NUM $\frac{\quad}{\text{Num}(n) \longrightarrow n}$ $\leftarrow$ fact, axiom

Language : CONST.

1+2+3.

Proof tree.

NUM $\frac{\quad}{1 \longrightarrow 1}$	NUM $\frac{\quad}{2 \longrightarrow 2}$	
ADD $\frac{\quad}{1+2 \longrightarrow 3}$	$\frac{\quad}{3 \longrightarrow 3}$	NUM.
ADD $\frac{\quad}{(1+2)+3 \longrightarrow 6}$		

Adding Decisions to the PL

IF TRUE $\frac{b = \text{true} \quad e_1 \rightarrow v_1}{\text{if } b \text{ then } e_1 \text{ else } e_2 \rightarrow v_1}$

IF FALSE $\frac{b = \text{false} \quad e_1 \rightarrow v_1 \quad e_2 \rightarrow v_2}{\text{if } b \text{ then } e_1 \text{ else } e_2 \rightarrow v_2}$

Classwork

- ① Extend the abstract syntax to allow non atomic boolean expressions.
- ② Define semantic rules to evaluate non-atomic booleans and branch expression with non-atomic conditional.
- ③ Extend the interpreter to evaluate expressions which have non-atomic boolean conditionals.

28/4/2026

Variables

$x + 10$

Variable: Context sensitive
element

↓
 x

Environments (σ)

$\sigma : \text{var} \longrightarrow \text{value}$

VAR $\frac{\sigma(x) = v}{\sigma \vdash x \longrightarrow v}$

ADD $\frac{\sigma \vdash e_1 \longrightarrow v_1 \quad \sigma \vdash e_2 \longrightarrow v_2}{\sigma \vdash e_1 + e_2 \longrightarrow v_1 + v_2}$

All rules to be rewritten to include σ .

let $x = 5$ in $x + 10$

Let $(x : \text{string}, e_1 : \text{expr}, e_2 : \text{expr})$

$$\text{Let } \frac{\sigma \vdash e_1 \rightarrow v_1 \quad \sigma[x \mapsto v_1] \vdash e_2 \rightarrow v_2}{\sigma \vdash \text{let } x = e_1 \text{ in } e_2 \rightarrow v_2}$$

$$\text{Let } \frac{\sigma \vdash e_1 \rightarrow v_1 \quad \begin{array}{l} \sigma' = \sigma[x \mapsto v_1] \\ \sigma' \vdash e_2 \rightarrow v_2 \end{array}}{\sigma \vdash \text{let } x = e_1 \text{ in } e_2 \rightarrow v_2}$$

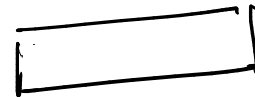
Expressions in reduced form / normal form.

let $x = 1$ in

let $y = 2$ in

let $x = x + y$ in
 $x + y$

s1



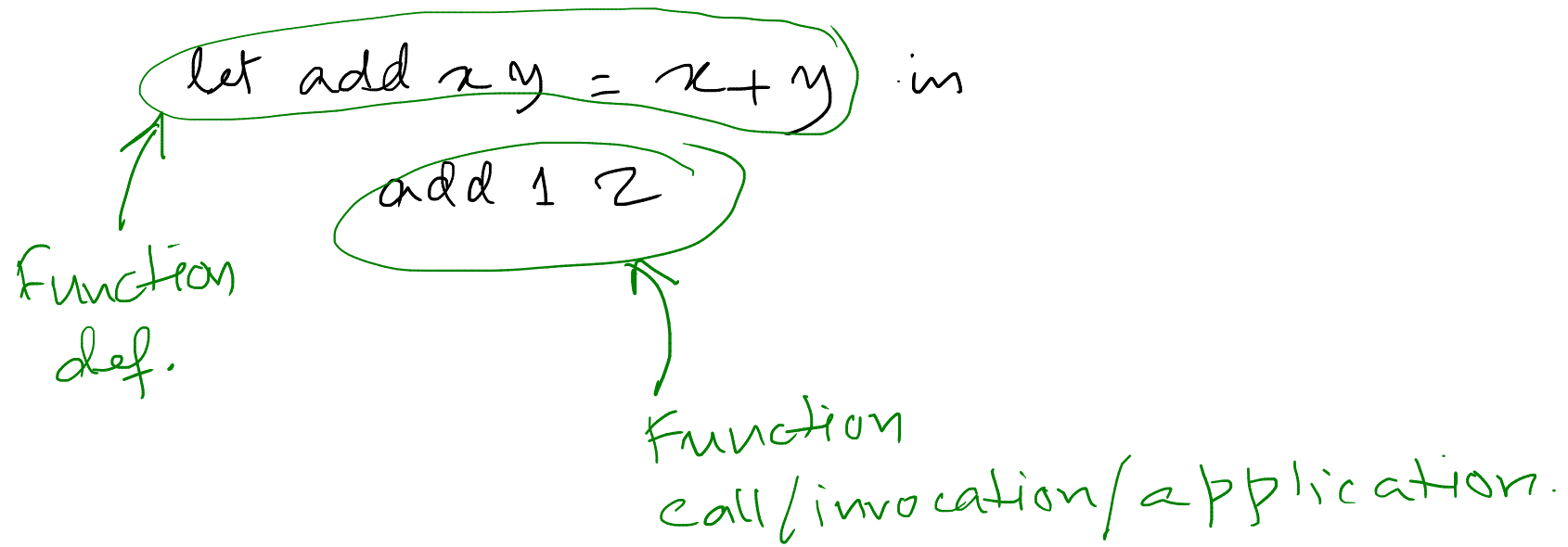
s2

let $x = 1$ in let $y = 2$ in let $x = x + y$ in $x + y$

$\Rightarrow [x \mapsto 1] \vdash \text{let } y = 2 \dots \dots$

$\Rightarrow [x \mapsto 1, y \mapsto 2] \vdash \text{let } x = x + y \dots \dots$

LET



let dbl $x = x + x$ in

dbl 2

let $x = 10$ in

let add $x\ y = x + y$ in

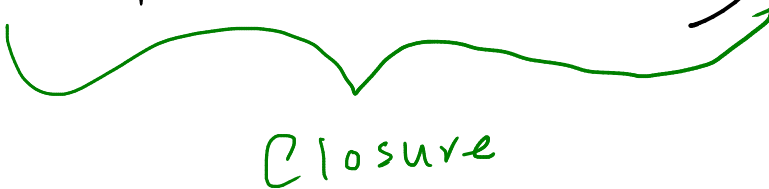
let $x = 30$ in

add $x\ 20$

Function Definition

- Function name
- Parameter
- Body

$f_{name} \rightarrow (\text{parameter}, \text{body}, \sigma)$



Closure